Graph Neural Networks

Every node updates itself by summarizing its neighbors

仲翊

MiRA Training Course 2026 · 9/3, R546

30 minutes, 3 ideas, and I stop at 17:50

1. Every node updates itself by summarizing its neighbors.
2. Layers are hops. Two or three is usually the right number.
3. The hard part is never the layer. It is deciding what the nodes and the edges are.

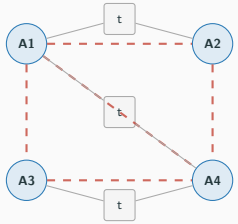
No derivations. One equation. No spectral graph theory.

Eight arms in one workspace, planned by a network

RoboBallet plans reaching motions for up to 8 arms and 40 tasks sharing one workspace, with a GNN plus RL and no per-scene re-planning.

Nodes: arms, tasks, obstacles.
Edges: who can collide with whom.

arXiv:2509.05397



dashed = possible collision

A robot that pours and folds by predicting particles

DPI-Net learns the dynamics of rigid bodies, deformable objects and fluids from particles, then uses that model to manipulate them.

Nodes: particles of the object.

Edges: contacts (and rigid-group links).

There is no mesh, no state vector, no fixed object count. Water is 3000 particles today and 5000 tomorrow, and the same network runs.

arXiv:1810.01566

A physics engine nobody wrote

GNS and **MeshGraphNets** simulate sand, water and cloth by message passing on the mesh, learned from data rather than written down as equations.

Nodes: particles, or mesh vertices.

Edges: within a contact radius; mesh edges.

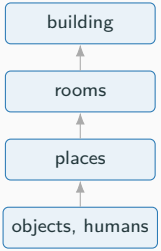
They generalize to more particles than they were trained on. Keep that fact. We come back to why.

arXiv:2002.09405

arXiv:2010.03409

A map a planner can reason over

3D Dynamic Scene Graphs turn a metric map into a graph of places, objects and humans: something a planner can query, not just a point cloud.



a hierarchy, so a planner can reason at the right scale

containment, adjacency, "the mug is on the table"

arXiv:2002.06289

All four are the same architecture

Eight arms. Fluid particles. A cloth mesh. A building.

One layer type, repeated.

Nothing above is a grid. Nothing above is a sequence.

What does a CNN assume? What does a Transformer assume?

Hands up. Shout it out.

What does a CNN assume? What does a Transformer assume?

Hands up. Shout it out.

CNN $\rightarrow$ a grid

neighbors are the pixels next to you

What does a CNN assume? What does a Transformer assume?

Hands up. Shout it out.

CNN $\rightarrow$ a grid

neighbors are the pixels next to you

Transformer $\rightarrow$ an ordered sequence

positions 1, 2, 3, ...

What does a CNN assume? What does a Transformer assume?

Hands up. Shout it out.

CNN $\rightarrow$ a grid

neighbors are the pixels next to you

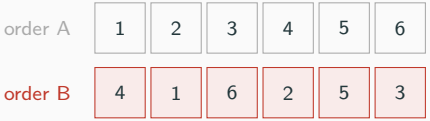
Transformer $\rightarrow$ an ordered sequence

positions 1, 2, 3, ...

None of the four things I just showed you is either one.

Flattening fails: relabel the nodes and the answer changes

Six objects on a table. Flatten them into one vector, feed an MLP.



Same scene. Different vector.
Different answer.

The MLP has to learn $n!$ copies of the same function. A GNN gets this for free, because summing over neighbors does not care about order.

Flattening fails: 8 objects today, 15 tomorrow

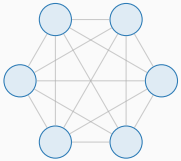
A fixed input dimension cannot take a variable number of things.

- 8 objects on the table today, 15 tomorrow.
- 3000 fluid particles in training, 5000 at test time.
- 4 robots in the lab, 9 at the demo.

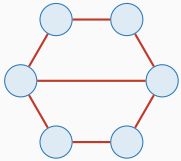
Padding and truncating is not a fix, it is a bug you have not hit yet.

“Isn’t attention already a graph?” Yes, a fully-connected one

A Transformer is a GNN on a complete graph,
with attention as the aggregation.



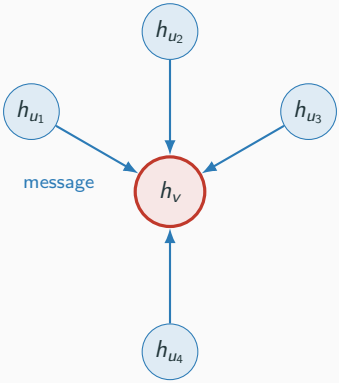
Transformer: *learn* the edges



GNN: you *already know* the edges

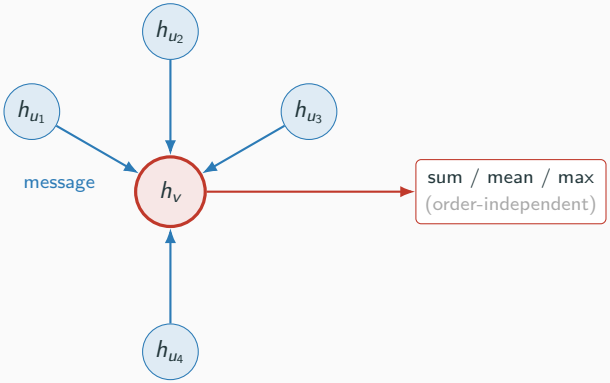
Which particles touch, which arms can collide. Don’t spend data and compute learning what you already know.

One idea: summarize your neighbors, then repeat



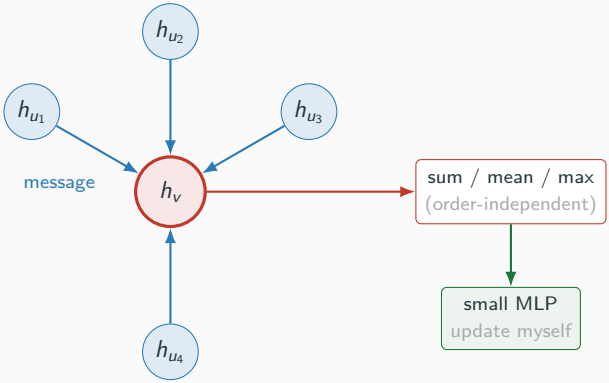
1. Every neighbor sends a message.

One idea: summarize your neighbors, then repeat



1. Every neighbor sends a message.
2. Combine them with something order-independent.

One idea: summarize your neighbors, then repeat



1. Every neighbor sends a message.
2. Combine them with something order-independent.
3. Update myself with a small MLP. Then repeat L times.

The message-passing equation, in three clicks

$$h_v \leftarrow \mathrm{UPDATE}\Big(h_v,\ \mathrm{AGG}\big\{\ \mathrm{MSG}(h_v, h_u, e_{uv}) :\ u \in \mathcal{N}(v)\big\}\Big)$$

One neighbor sends a message. It may use my features, its features, and *what is on the edge between us*.

The message-passing equation, in three clicks

$$h_v \leftarrow \mathrm{UPDATE}\Big(h_v, \mathrm{AGG}\big\{\, \mathrm{MSG}(h_v, h_u, e_{uv}) : \; u \in \mathcal{N}(v)\big\}\Big)$$

Combine all the messages. Sum, mean or max: anything that ignores the order. This is where permutation invariance comes from.

The message-passing equation, in three clicks

$$h_v \leftarrow \mathrm{UPDATE}\Big(h_v, \mathrm{AGG}\big\{\, \mathrm{MSG}(h_v, h_u, e_{uv}) \;:\; u \in \mathcal{N}(v) \big\}\Big)$$

Update myself. A small MLP on (me, the summary). That is the entire layer.

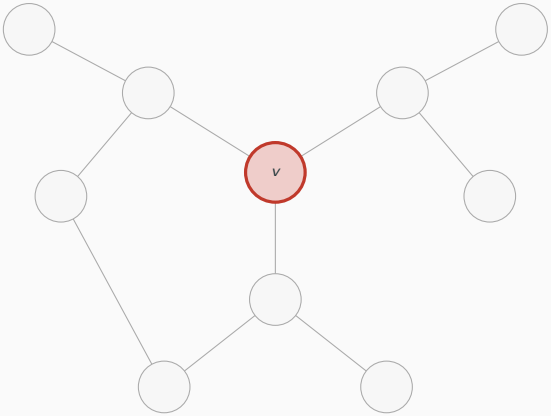
The message-passing equation, in three clicks

$$h_v \leftarrow \mathrm{UPDATE}\Big(h_v, \mathrm{AGG}\big\{\, \mathrm{MSG}(h_v, h_u, e_{uv}) \;:\; u \in \mathcal{N}(v) \big\}\Big)$$

Stack L of these. That is a GNN. There is nothing else.

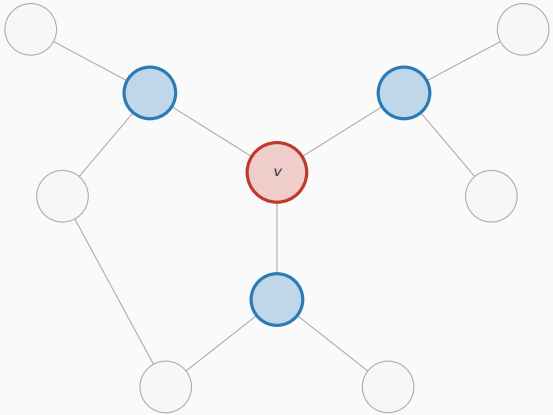
GCN in one line: $H \leftarrow \hat{A}\,(HW)$, with $\hat{A} = D^{-1/2}(A + I)D^{-1/2}$, a normalized neighbor average, then a linear map.

L layers = an L-hop receptive field



layer 0: just me

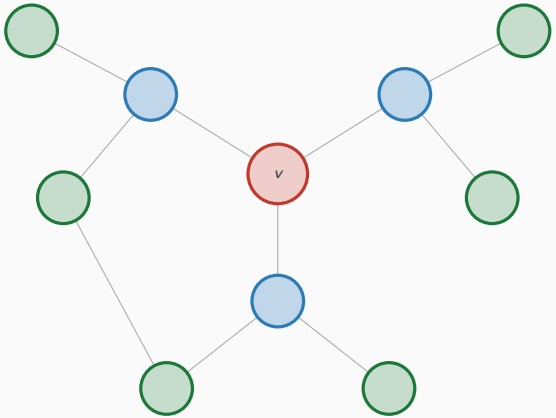
L layers = an L-hop receptive field



layer 0: just me

after 1 layer: 1 hop

L layers = an L-hop receptive field

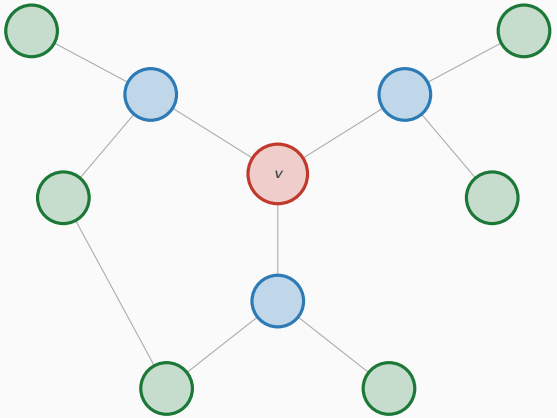


layer 0: just me

after 1 layer: 1 hop

after 2 layers: 2 hops

L layers = an L-hop receptive field



layer 0: just me

after 1 layer: 1 hop

after 2 layers: 2 hops

Same picture as a CNN's receptive field. Layers are hops.

GCN, GraphSAGE, GAT differ only in the AGG box

	AGG	Use it when
GCN	normalized mean	always. This is your baseline
GraphSAGE	sample neighbors, then mean/max	the graph is large, or new nodes appear at test time
GAT	attention-weighted sum	neighbors should not count equally

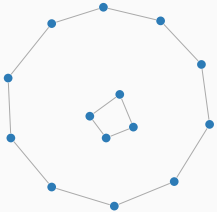
Three famous papers. One box changed.

Three task types, three readouts

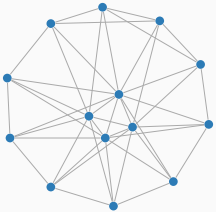
- **Node-level**: read off h_v.
  Label every point in the cloud. Predict every particle's next position.
- **Edge-level**: read off a function of (h_u, h_v).
  Will this pair collide? Is this a loop closure?
- **Graph-level**, which needs a readout: sum or mean over all nodes, then an MLP.
  Is this scene graspable? Will this grasp succeed?

This is the slide that maps *your* problem onto a GNN. Pick your row first.

You will spend 90% of your time on graph construction



$k = 3$: *two disconnected pieces*



$k = 20$: *structure thrown away*

Same 225 points, three different models:

k	edges	mean degree
3	421	3.7
8	1054	9.4
20	2545	22.6

k-NN vs. radius vs. fully-connected. How many neighbors.

And in robotics: put the relative pose on the edge. That buys translation invariance for free, instead of paying for it in training data.

What the nodes and edges actually were

Application	Nodes	Edges	Paper
Multi-arm motion planning	arms, tasks, obstacles	possible collisions	RoboBallet arXiv:2509.05397
Deformables and fluids	particles of the object	contact + rigid-group	DPI-Net arXiv:1810.01566
Learned physics	particles	within a radius	GNS arXiv:2002.09405
Learned physics (mesh)	mesh vertices	mesh edges	MeshGraphNets arXiv:2010.03409
Deformables from vision	object keypoints	learned latent graph	arXiv:2104.12149
Point clouds	points	k-NN, recomputed *in feature space*	DGCNN / EdgeConv arXiv:1801.07829
Grasp pose detection	points of the cloud	local neighborhoods	Grasp-the-Graph 2.0 arXiv:2505.02664
Multi-robot planning	robots	communication range (decentralized)	Li & Prorok arXiv:1912.06095
Spatial perception	places, objects, agents	containment, adjacency	3D Dynamic Scene Graphs arXiv:2002.06289

This is the column you copy into your own project.

In multi-robot systems, a “message” is a real radio message

Nodes are robots. Edges are who is in communication range.

One round of message passing = one round of radio traffic.

- So the number of layers is a bandwidth decision, not a hyperparameter.
  arXiv:1912.06095
- And in physics and manipulation, the graph gives you the right invariance for free: particles interact locally and identically, which is exactly what a shared message function assumes.
- That is why learned simulators generalize to more particles than they were trained on.

Deeper is worse: over-smoothing after 3 layers

layers	test acc.	mean pairwise dist.
1	71.2%	
2	91.3%	
3	**96.9%**	
4	96.2%	
5	96.2%	5.6
6	**54.4%**	**0.10**
7	56.2%	
8	48.8%	

After enough rounds of averaging with your neighbors, every node in a connected graph holds the same vector. That is over-smoothing, and it is a property of the averaging, not of the task.

Two honest caveats:

- The cliff mixes over-smoothing with the plain difficulty of training deep GCNs without residual connections.
- This is a toy task with a *global* label, so extra hops keep helping longer than usual. On real benchmarks the optimum is 2–3 layers.

The rule survives either explanation: start at 2.

A complete GNN in 12 lines of PyTorch Geometric

```
from torch_geometric.nn import GCNConv
from torch_geometric.data import Data

data = Data(x=X, edge_index=A.nonzero().t().contiguous(), y=y)

class GCN(nn.Module):
    def __init__(self, d_in, d_hid, d_out):
        super().__init__()
        self.c1, self.c2 = GCNConv(d_in, d_hid), GCNConv(d_hid, d_out)

    def forward(self, d):
        h = F.relu(self.c1(d.x, d.edge_index))
        return self.c2(h, d.edge_index)
```

That is the whole model. Everything hard happened on line 4, where you chose the edges.

Run it yourself: everything above, on a free Colab CPU

colab.research.google.com/github/Chung-I/MiRA_training_course_2026/blob/main/notebooks/gnn_demo.ipynb

PyTorch and NumPy only. Message passing is written out by hand so you can read every line. PyTorch Geometric appears once, at the end, optional.

Right now, live:

- Demo 4: node features are *pure noise*
- Demo 5: k-NN at $k = 3, 8, 20$

Demo 4: two communities, 200 nodes, features carry *no* information

MLP on features only	51.2%
2-layer GCN	**91.3%**

The graph is the entire signal.

When *not* to use a GNN

- **Small and densely connected?** Just use a Transformer. It is simpler, better tooled, and on a complete graph it is the same thing anyway.
- **No real structure?** If you had to invent the edges to make it a graph, you invented a prior. It may be wrong.
- **Your data is a grid or a sequence?** You already have the right architecture.

GNNs win when the structure is sparse, known, and large.

Is my problem a graph? A 3-question checklist

1. Do I have entities with relations between them?
2. Is there no natural ordering of those entities?
3. Does the number of them change?

Two yeses $\Rightarrow$ try a GNN.

Take-home

A GNN is one idea repeated: every node updates itself by summarizing its neighbors.

The hard part is never the layer. It is deciding what the nodes and the edges are.

Where to learn more

- Stanford **CS224W**, the course, if you want the full picture
- The **PyTorch Geometric** `examples/` directory, the fastest way to a working model
- Distill, *A Gentle Introduction to Graph Neural Networks*, one afternoon, all pictures
- Battaglia et al., arXiv:1806.01261, the position paper, if you want the framing in full

Questions?

仲翊 · MiRA Training Course 2026

Slides and notebooks: `github.com/Chung-I/MiRA_training_course_2026`